SVKM's NMIMS
Mukesh Patel School of Technology Management and Engineering, Mumbai

NavCert Offline

An Offline Android Application for Scanning and Reading Maritime Certificates

---

Mini Project Report

Mobile Application Development (702AI0E002)

Submitted by: Aadidev Raizada

Roll No.: N152

B.Tech, Semester V

SVKM's NMIMS Mukesh Patel School of Technology Management and Engineering, Mumbai

Academic Year 2026-27

Index

Chapter and section **Page**

1 Introduction....3
1.1 Background....3
1.2 Problem Statement....3
1.3 Objectives....3
1.4 Scope....4
2 User Requirements....4
2.1 Target Users and Their Needs....4
2.2 Functional Requirements....4
2.3 Non-Functional Requirements....5
3 Tools and Technologies....5
4 System Design....6
4.1 Architecture Overview....6
4.2 Module Description....6
4.3 Data Flow....7
4.4 Navigation and Screens....7
4.5 Storage Design....7
4.6 Permissions....8
5 Implementation....8
5.1 Capture and Crop....8
5.2 Preview and Enhancement....9
5.3 Text and Certificate Fields....9
5.4 Document Library and Ask....10
5.5 Pipeline Lab....10
5.6 Navigation and Privacy....11
6 Mobile Development Concepts Applied....12
7 Advanced Features....12
7.1 On-Device ML Runtimes....13
7.2 Fine-Tuned MiniLM Field Model....13
7.3 OpenCV Document Pipeline....13
7.4 Hybrid Field Extraction....13
7.5 Coroutines and Dispatchers....14
7.6 Activity Result API and FileProvider....14
8 Design Guidelines and Release Readiness....14
8.1 Material Design 3....14
8.2 Play Store Release Checklist....15
9 Testing and Results....15
9.1 Unit Tests....15
9.2 Device and Integration Testing....15
9.3 Model Results....16
9.4 Not Yet Measured....16
10 Project Work and Timeline....16
10.1 Individual Work....16
10.2 Timeline....17
10.3 Problems Solved....17
11 Conclusion and Future Scope....17
11.1 Conclusion....17
11.2 Future Scope....18
References....18

1 Introduction

1.1 Background

Merchant shipping depends on certificates. Every seafarer carries certificates of competency, proficiency and medical fitness, and every ship carries certificates of registry, safety and insurance. The BIMCO and ICS Seafarer Workforce Report 2026 estimates that 2,565,580 STCW-certified seafarers serve the world merchant fleet [1]. These papers are checked at every stage of a seafarer's career: by crewing agents before a contract, by ship managers before joining, and by port state control inspectors on board.

The checks find real problems. The Tokyo MOU recorded 877 crew-certificate deficiencies in 2022, 920 in 2023 and 1,304 in 2024 [2]. In India, about 80% of seafarers work on foreign-flagged ships, and a Directorate General of Shipping circular of July 2025 requires recruitment and placement agencies to verify the authenticity of all certificates [3]. At the same time, the IMO guidelines on electronic seafarer certificates assume that the verifying party "should at least have an Internet connection" [4].

In practice, certificates travel as phone photographs and scans, taken at an angle and in poor light, on ships and in ports where the connection is weak or expensive. A generic scanner app can straighten the picture, but it does not know what an issue date or a port of issue is, and most document apps send the image to a cloud service for text recognition.

1.2 Problem Statement

Seafarers, crewing agents and ship managers keep certificates as scattered photos, scans and paper. Key dates are hard to find, the same fields are retyped for every application, and cloud scanners need a connection and expose identity documents to third parties. This project asks whether an Android phone can capture a certificate, read it, extract its key fields and answer questions about it with no network access at all.

1.3 Objectives

O1 Capture a certificate with the camera or from the gallery and turn a skewed photo into a flat, clean page on the device.

O2 Read printed and handwritten text on the device, with no download at first use.

O3 Extract five fields: certificate name, issue date, port of issue, issuing authority and expiry date, without inventing values for documents that are not certificates.

O4 Save documents locally with search, folders, editing, notes and highlights.

O5 Answer natural-language questions about saved documents offline and show the source document.

O6 Follow Material Design and prepare the app for release on Google Play, with no network permission.

1.4 Scope

The app targets Android phones and tablets from Android 7.0 (API 24) and is built against API 36. It handles English-language certificates in Latin script. All processing and storage happen on the device. The app reads certificates; it does not verify their authenticity against an issuing registry, because that needs a network connection by definition. Accounts, cloud sync and expiry reminders are outside the current scope and are listed as future work in Chapter 11.

2 User Requirements

2.1 Target Users and Their Needs

Three user groups were identified from the sourced facts in Chapter 1 and from the certificate corpus of 108 PDFs prepared for this project. They were not derived from interviews. Table 2.1 links each need to the feature that answers it.

Table 2.1 User groups, needs and the features that address them

User	Need	Feature in the app
Seafarer	Keep identity documents private; work with no signal at sea	No INTERNET permission; app-private storage; backup disabled
Crewing agent	Stop retyping the same five fields for every candidate	Certificate name, issue date, port, authority and expiry extracted automatically
Ship manager	Find a document and its expiry date quickly	Library search, folders, Ask screen that links to the source document
Any user	Turn a crooked photo into a clean page	Crop editor, OpenCV page detection, perspective correction and enhancement
Any user	Trust and correct the output	Editable text, notes and highlights; no dates invented on non-certificates

2.2 Functional Requirements

FR1 Capture an image with the system camera or choose one with the system picker.

FR2 Let the user crop and rotate the image before processing.

FR3 Detect the page, correct its perspective, enhance it (Auto, Clean or B&W) and report image quality.

FR4 Recognise text in three modes: Auto, Printed and Handwriting.

FR5 Extract the five certificate fields and skip extraction for documents that are not certificates.

FR6 Allow editing of the text, a title, review notes, highlights and a bookmark before saving.

FR7 Save, list, search, move into folders and delete documents.

FR8 Answer a question from all saved documents or one folder and open the source document.

FR9 Copy or share text only when the user asks; show what the app does with data.

2.3 Non-Functional Requirements

NFR1 Privacy and offline operation: no network permission, no account, no analytics; every model is bundled in the app.

NFR2 Responsiveness: image processing and model inference run off the main thread, with a progress indicator in the interface.

NFR3 Compatibility: minimum SDK 24, target SDK 36; phone and tablet layouts.

NFR4 Reliability: documents are written atomically so a crash cannot leave a half-written file.

NFR5 Usability: Material 3 components, content descriptions on icon buttons, clear status messages.

NFR6 Trustworthiness: extracted values must come from the document text; the user can correct them.

NFR7 Maintainability: pure Kotlin logic covered by JVM unit tests that run without a device.

3 Tools and Technologies

The single constraint of no network access shaped every choice: each library had to run fully on the device and each model had to ship inside the app. The versions in Table 3.1 are those declared in `app/build.gradle.kts` and `gradle/libs.versions.toml`, the two Gradle files of the project.

Table 3.1 Tools, versions and purpose

Tool or library	Version	Purpose in this project
Kotlin with Jetpack Compose [5]	Kotlin 2.0.21; Compose BOM 2024.09.00	Declarative, state-driven user interface
Material 3 for Compose [6]	from the Compose BOM	Standard components, colour scheme and typography
Android Gradle Plugin; Android SDK	AGP 9.0.1; minSdk 24, target and compile SDK 36	Build, R8 release shrinking, Play target level
OpenCV [7]	4.13.0	Page detection, perspective warp, enhancement, quality checks
ML Kit Text Recognition [8]	16.0.1	Printed text with the bundled Latin model; line boxes
ONNX Runtime for Android [9]	1.24.1	PP-OCRv5 handwriting recognition [10]; fine-tuned MiniLM field model
TensorFlow Lite Task Text [11]	0.4.4	MobileBERT question answering [12]
Coil; Android Image Cropper	2.7.0; 4.7.0	Library thumbnails; crop and rotate editor
AndroidX core-ktx, lifecycle, activity-compose	1.10.1, 2.6.1, 1.8.0	Activity Result API, lifecycle-aware coroutines

Development used Android Studio with the Gradle Kotlin DSL, the Android emulator and an OPPO CPH2665 phone running Android 15. Unit tests use JUnit 4.13.2. The

dataset preparation, MiniLM fine-tuning, int8 export and evaluation are Python scripts in `ml/`. Large model files are tracked with Git LFS. The bundled models are PP-OCRv5 mobile recognition (7.6 MiB, `.ort` format), the fine-tuned MiniLM (32.3 MiB, int8 ONNX) and MobileBERT (95.8 MiB, TensorFlow Lite).

4 System Design

4.1 Architecture Overview

The app has three layers inside one device boundary (Fig. 4.1). The presentation layer is a single `MainActivity` that hosts the Compose app and a navigation drawer, plus `CropEditorActivity`, which opens through an explicit intent and returns the cropped image. The processing pipeline runs in eight steps on background coroutines. The local resources layer holds app-private files, the camera cache shared through a `FileProvider`, and the bundled models. Because the network permissions are removed in the manifest, Android blocks every connection from the app.

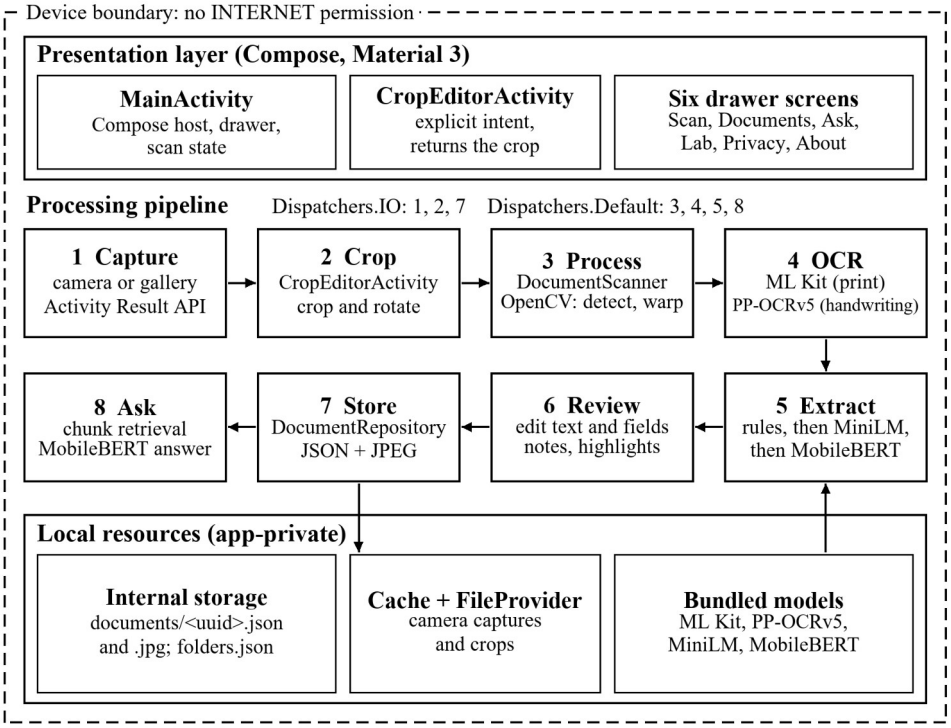


Fig. 4.1 Architecture of NavCert Offline

4.2 Module Description

Table 4.1 Modules and source files

Module	File	Responsibility
Activities	`MainActivity.kt`, `CropEditorActivity.kt`	Compose host, drawer, scan state machine; crop editor
Screens	`ScanWorkflowScreens.kt`, `AppScreens.kt`, `PipelineLabScreen.kt`	Capture, preview, result; documents, ask, privacy, about; Pipeline Lab

Module	File	Responsibility
Vision	`DocumentScanner.kt`	Quality check, page detection, warp, enhancement (OpenCV)
OCR	`PaddleHandwritingRecognizer.kt`, `DocumentTextFormatter.kt`, `OcrTextCorrector.kt`	PP-OCRv5 lines; layout rebuilt from boxes; conservative correction
Understanding	`CertificateFieldExtractor.kt`, `CertificateOnnxAnswerer.kt`, `OfflineDocumentAssistant.kt`	Rules, MiniLM and MobileBERT; retrieval and answers
Storage	`DocumentRepository.kt`	JSON and JPEG per document; folders; atomic writes
Pipeline Lab	`IvpOperations.kt`, `PipelineLabProcessor.kt`	Each image-processing stage on a scan or a bundled sample
Theme	`ui/theme/Color.kt`, `Theme.kt`, `Type.kt`	Material 3 colour scheme and typography

4.3 Data Flow

1 **Capture.** `TakePicture` writes a camera photo to a cache file exposed through the `FileProvider`; `GetContent` returns a gallery URI. The bitmap is decoded on `Dispatchers.IO`.

2 **Crop.** If requested, `CropEditorActivity` returns a cropped URI through `StartActivityForResult`.

3 **Process.** `DocumentScanner` scales the image (at most 1600 px for analysis, 3200 px for OCR), checks quality, finds the page, warps it flat and enhances it on `Dispatchers.Default`.

4 **OCR.** ML Kit reads printed text and returns line boxes. In Handwriting mode each line is re-read by PP-OCRv5 on ONNX Runtime, with ML Kit as the fallback.

5 **Extract.** `CertificateFieldExtractor` fills the five fields (Chapter 7).

6 **Review.** The user edits the text and fields, adds notes and highlights, and saves.

7 **Store.** `DocumentRepository` writes the JSON record and a JPEG preview.

8 **Ask.** Saved documents are split into chunks, the best chunks are retrieved and MobileBERT returns an answer with its source document.

4.4 Navigation and Screens

Navigation uses an `AppDestination` enum with six drawer entries: Scan, Documents, Ask, Pipeline Lab, Privacy & offline, and About. The Scan destination has its own sub-flow, `ScanStep`: CAPTURE, PREVIEW and RESULT, and `BackHandler` steps back through it. The drawer is a `ModalNavigationDrawer` on phones and switches to a `PermanentNavigationDrawer` when the window is at least 840 dp wide, so tablets show a permanent side menu.

4.5 Storage Design

Each saved document is a JSON file at `files/documents/<uuid>.json` with its text, title, timestamps, word, line and character counts, review notes, highlighted phrases,

bookmark, folder, source type and recognition mode. Its preview is a JPEG at quality 88 beside it, and folders are kept in `files/folders.json`. Every write goes to a temporary file that is then renamed, so a document is either fully saved or not saved. The ONNX models are copied once from assets into `files/models/` because ONNX Runtime needs a file path. All of this is internal app storage that other apps cannot read [13].

4.6 Permissions

Table 4.2 Permissions and how each need is met

Permission	Status	How the need is met
INTERNET; ACCESS_NETWORK_STATE	Removed	ML Kit's dependencies request them; `tools:node="remove"` strips them in the merged manifest
CAMERA	Not requested	The system camera app captures into a `FileProvider` URI [14]
Storage, photos and media	Not requested	The system picker grants access to the chosen image only
Location	Not used	The problem does not need the user's position

Because the app asks for no dangerous permission, it never shows a runtime permission dialog [15]. Backup is disabled with `android:allowBackup="false"`, and the backup and data-extraction rules exclude all app data from cloud backup and device-to-device transfer.

5 Implementation

This chapter walks through the app screen by screen. All screenshots show a synthetic certificate made by the script `tools/make_sample_certificate.py` and were taken on the Android emulator. No real certificate appears in this report.

5.1 Capture and Crop

The Scan screen (Fig. 5.1a) offers two buttons, Open camera and Choose an image, and a text-type choice of Auto, Printed or Handwriting made with Material filter chips. Choose an image opens the system photo picker (Fig. 5.1b), which tells the user that the app will only see the photo they select. The crop editor (Fig. 5.1c) is a second activity that hosts the cropper view through `AndroidView`; the user drags the corners, rotates and applies the crop.

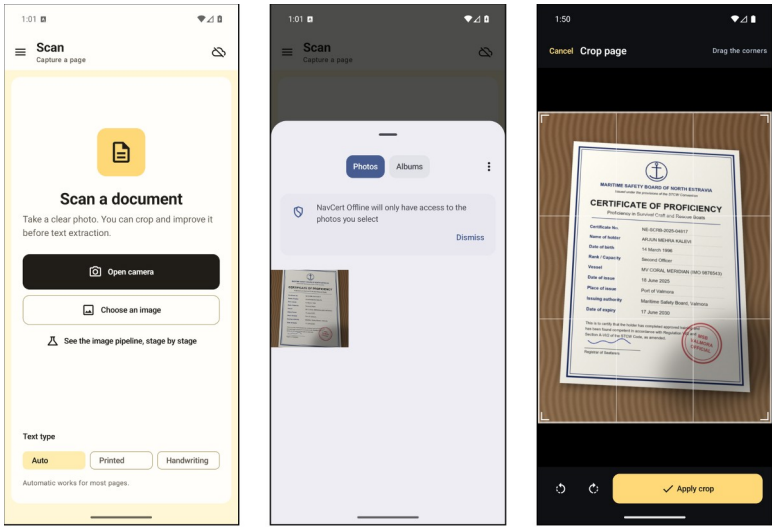


Fig. 5.1 Capture and crop: (a) Scan screen, (b) system photo picker, (c) crop editor

5.2 Preview and Enhancement

After capture, `DocumentScanner` detects and flattens the page. The preview (Fig. 5.2a) shows the aligned page, a quality hint (here, Image looks good), Crop and Rotate buttons and three enhancement modes: Auto, Clean and B&W. The quality hint flags a photo as too dark (mean gray below 55), too bright (above 220) or soft (Laplacian variance below 65). Fig. 5.2b and 5.2c show two of the steps behind the preview in the Pipeline Lab: the detected page corners with the edge map used to find them, and the input page beside global histogram equalisation and CLAHE.

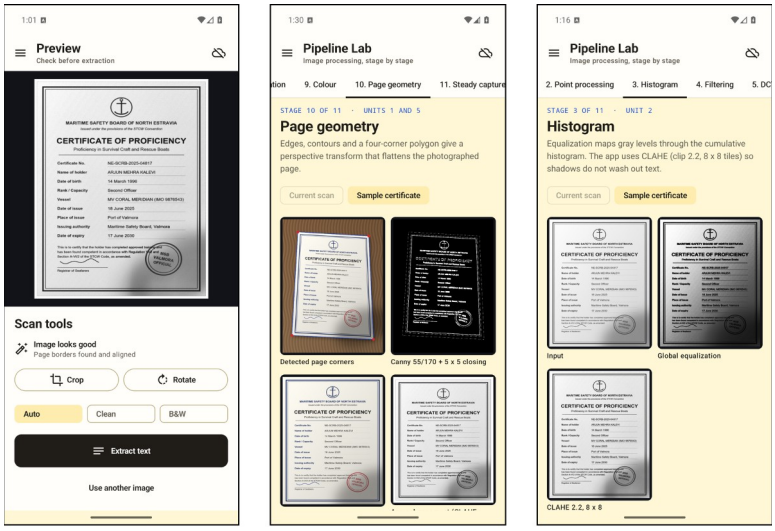
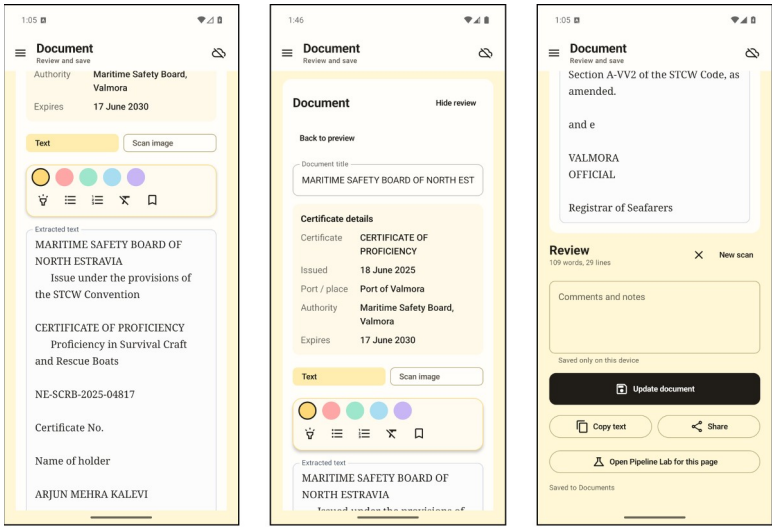


Fig. 5.2 Preview and enhancement: (a) aligned page with scan tools, (b) page geometry stage, (c) histogram stage

5.3 Text and Certificate Fields

Extract text runs OCR and field extraction. The result screen (Fig. 5.3a) shows the recognised text with tools for highlight colours, list formatting and bookmarks. Above the text, a Certificate details card (Fig. 5.3b) shows the five extracted fields; for the sample

they read Certificate of Proficiency, issued 18 June 2025 at the Port of Valmora by the Maritime Safety Board, Valmora, expiring 17 June 2030. The review panel (Fig. 5.3c) holds review notes and the Update document, Copy text, Share and Open Pipeline Lab actions. Share uses an implicit `ACTION_SEND` intent through the Android share sheet.

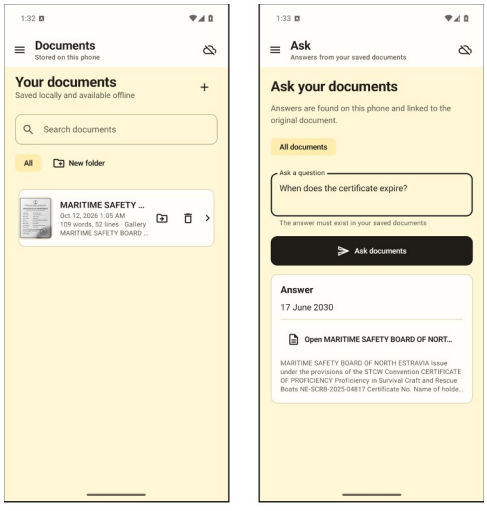


(a) (b) (c)

Fig. 5.3 Text and fields: (a) extracted text, (b) certificate details card, (c) review and save

5.4 Document Library and Ask

The Documents screen (Fig. 5.4a) lists saved documents with a thumbnail loaded by Coil, a search box, folder chips, a New folder action, and move and delete buttons that open alert dialogs. The Ask screen (Fig. 5.4b) takes a question such as When does the certificate expire?, searches all documents or one folder, and shows the answer (17 June 2030) with a link that opens the source document and the passage it came from.



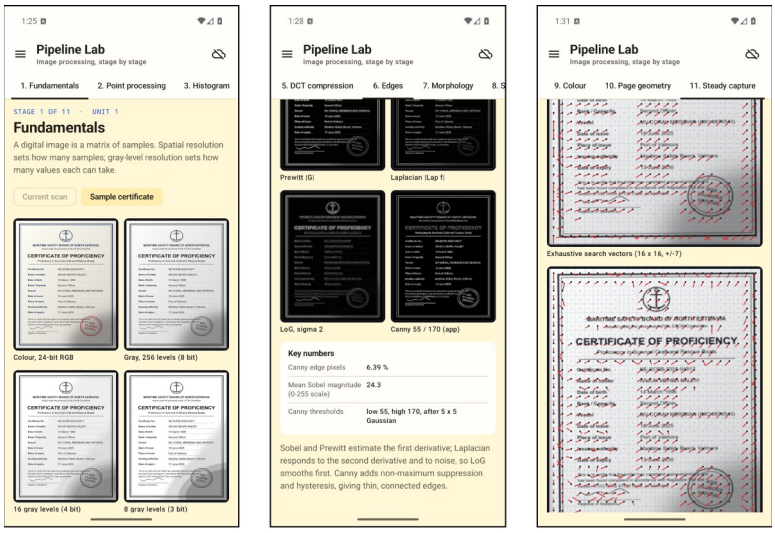
(a) (b)

Fig. 5.4 Library and Ask: (a) Documents screen, (b) Ask screen with answer and source

5.5 Pipeline Lab

The Pipeline Lab shows each image-processing stage of the app on the current scan or on a bundled synthetic certificate, so the pipeline can be explained without a camera or

private data. It has eleven stages in a scrollable tab row: fundamentals, point processing, histogram, filtering, DCT compression, edges, morphology, segmentation, colour, page geometry and steady capture. Each stage shows the images and a table of key numbers, and some stages have sliders (for example JPEG quality and gamma) and charts drawn with the Compose `Canvas`. The operations are pure Kotlin in `IvpOperations.kt` and `PipelineLabProcessor.kt`.

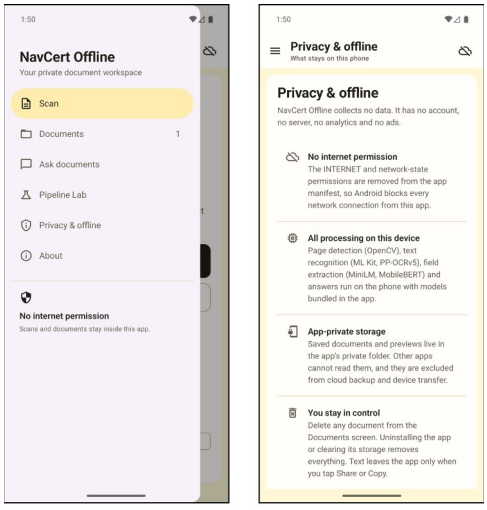


(a) (b) (c)

Fig. 5.5 Pipeline Lab: (a) fundamentals, (b) edges with Canny, (c) steady capture with block-matching vectors

5.6 Navigation and Privacy

The navigation drawer (Fig. 5.6a) lists the six destinations with Material icons and a count of saved documents. The Privacy & offline screen (Fig. 5.6b) states in plain language that the app has no internet permission, that all processing runs on the device, that documents live in app-private storage excluded from backup, and how the user can delete them. The same text is published as `docs/PRIVACY.md`, which serves as the privacy policy.



(a) (b)

Fig. 5.6 Navigation and privacy: (a) navigation drawer, (b) Privacy & offline screen

6 Mobile Development Concepts Applied

The course is organised in seven units with four course outcomes (CO1 to CO4). Table 6.1 maps each unit to the app and states plainly where a unit is not used.

Table 6.1 Course units mapped to the app

Unit and CO	Where it appears in the app	Status
1 Development platform (CO1)	Android Studio project, Gradle build, emulator and phone runs	Used
2 UI components (CO1)	Buttons, filter chips as choice controls, navigation drawer as menu, three alert dialogs (new folder, move, delete), two activities and six screens, Canvas graphics, sliders, back handling. No date picker, spinner, check box, radio button or toast is used.	Used
3 Images and files (CO2)	Camera and gallery images, FileProvider, JSON text files and JPEG previews in private storage, model files copied from assets	Used
4 Location tracking (CO3)	Not used: the app is offline and never needs a position	Not used
5 Flutter (CO3)	Not used: native Kotlin gives direct access to OpenCV, ONNX Runtime and TensorFlow Lite	Not used
6 Databases (CO3)	Not used: documents are stored as files; Firebase needs a network	Not used
7 Publishing (CO1, CO4)	No server by design; release build with R8 verified; not yet published on Google Play	Partly

Units 1 to 3. These units form the core of the app. Unit 2 appears in every screen, built with Compose equivalents of the classic controls: filter chips replace radio buttons for the text-type and enhancement choices, the drawer is the menu, and `AlertDialog` provides pop-up dialogs. Multiscreen navigation combines drawer destinations, the scan sub-flow and a second activity. Unit 3 covers the whole capture and storage path: displaying images, using images stored on the device, and reading and writing text files.

Unit 7. The client-server part does not apply because the app has no server by design. The publishing part is partly done: the release build type enables R8 minification and resource shrinking, the build was verified on the emulator, and the remaining Play steps are listed in Chapter 8.

Units 4, 5 and 6. Location tracking was left out because the app has no network and no reason to know where the user is. Flutter was not used because the project needs native libraries that are most direct to call from Kotlin. A database was not needed for the first version, since each document is a self-contained JSON file. A Room database for the document index [16] is the planned way to cover Unit 6, and it would also enable full-text search.

7 Advanced Features

Six features go beyond the course syllabus. Each runs in the app and is implemented in a named file.

7.1 On-Device ML Runtimes

The app runs three on-device inference stacks. ML Kit Text Recognition reads printed text with its bundled Latin model, so there is no first-use download [8] (`MainActivity.kt`). ONNX Runtime runs the PP-OCRv5 mobile recogniser [10] for handwriting (`PaddleHandwritingRecognizer.kt`). That file crops each ML Kit line box, resizes it to a height of 48 px, packs it into a tensor and decodes the output with greedy CTC decoding [17]. When the model fails or its mean confidence is below 0.20, the ML Kit text for that line is kept. TensorFlow Lite runs MobileBERT [12] through the Task Library `BertQuestionAnswerer` (`OfflineDocumentAssistant.kt`).

7.2 Fine-Tuned MiniLM Field Model

The certificate field model started from `deepset/minilm-uncased-squad2` [18], [19], an extractive question-answering model trained on SQuAD 2.0 [20]. The author fine-tuned it on certificate text from the project corpus and exported it to ONNX with int8 quantisation [21], which reduced it to 32.3 MiB. `CertificateOnnxAnswerer.kt` contains a WordPiece tokenizer written in Kotlin, runs the model over 384-token windows with a stride of 96, and picks the best answer span of up to 32 tokens. The model session is created lazily under a lock. On the human-checked gold set the int8 model answers 15 of 26 fields correctly, against 4 of 26 for the first prototype (Chapter 9).

7.3 OpenCV Document Pipeline

`DocumentScanner.kt` converts the image to gray, applies a 5 x 5 Gaussian blur and Canny edge detection [22] with thresholds 55 and 170, closes gaps with a 5 x 5 morphological close, and takes the largest four-cornered contour covering at least 15% of the image as the page. A perspective warp flattens it. Enhancement then applies CLAHE [23] with clip limit 2.2 on 8 x 8 tiles, a 3 x 3 median filter, and either an unsharp mask (Auto and Clean) or adaptive thresholding with block size 35 and constant 11 (B&W), and adds an 18 px white border for OCR.

7.4 Hybrid Field Extraction

`CertificateFieldExtractor.kt` puts deterministic rules first and uses a model only when the document gives evidence for the field (Fig. 7.1). A certificate gate checks the first 1,500 characters for words such as certificate, licence or document of compliance; if none is found, no fields are returned. Rules then read labelled values and patterns such as "issued at ... on ...", and ignore dates attached to batteries or equipment. Only missing fields are asked of the MiniLM model, with MobileBERT as the fallback if ONNX Runtime fails. A model answer for a date is accepted only if it contains a valid date.

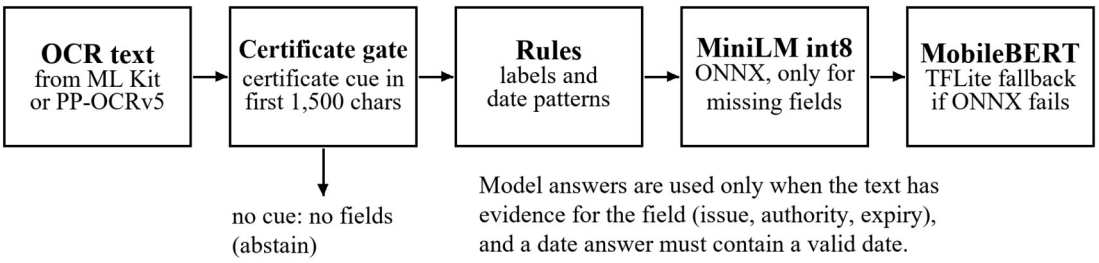


Fig. 7.1 Hybrid field extraction flow

7.5 Coroutines and Dispatchers

Heavy work is started from a coroutine scope (`rememberCoroutineScope()`) in `MainActivity.kt`. File decoding and repository reads and writes use `Dispatchers.IO`; rotation, OpenCV processing, PP-OCRv5 inference, field extraction and question answering use `Dispatchers.Default`. Native resources such as the ML Kit client, ONNX sessions, the TensorFlow Lite answerer and large bitmaps are released in `DisposableEffect` blocks, and MobileBERT is loaded lazily on first use.

7.6 Activity Result API and FileProvider

Three launchers are registered in `MainActivity.kt` with `rememberLauncherForActivityResult`. `TakePicture` opens the camera, `GetContent` opens the picker and `StartActivityForResult` opens the crop editor. The camera writes into a cache file exposed by the `FileProvider` declared in `AndroidManifest.xml` with a cache-only path in `res/xml/file_paths.xml`, so the app needs no camera or storage permission. The same manifest uses manifest-merger rules to remove the network permissions that library dependencies request.

A seventh, smaller feature supports the Ask screen: `OfflineDocumentAssistant.kt` splits documents into 1,200-character chunks with 180 characters of overlap and ranks them with TF-IDF-style weights before MobileBERT reads the best chunks.

8 Design Guidelines and Release Readiness

8.1 Material Design 3

Every screen is built from Material 3 components [24], [6]: `Scaffold` with a `TopAppBar`, `NavigationDrawerItem` entries, `FilterChip` choices, filled, outlined and text buttons, `OutlinedTextField` inputs, `Card` containers, `AlertDialog`, `LazyColumn`, a scrollable tab row and `LinearProgressIndicator` during processing. A custom light colour scheme and typography are defined in `ui/theme`. The layout adapts to width: the drawer is modal on phones and permanent from 840 dp. Icon-only buttons carry content descriptions for screen readers, and status text explains what the app is doing and what went wrong. The crop editor draws edge to edge and respects system-bar insets. The main screens support the light theme only for now.

8.2 Play Store Release Checklist

Google Play requires new apps and updates to target Android 16 (API level 36) or higher from 31 August 2026 [25], and a Data safety form is required even for apps that collect no user data [26]. Table 8.1 records the state of each release item.

Table 8.1 Release checklist

Item	Status	Evidence or next step
App name and adaptive launcher icon	Done	`NavCert Offline` in `strings.xml`; `mipmap-anydpi-v26/ic_launcher.xml`
Target SDK 36, minimum SDK 24	Done	`app/build.gradle.kts`
No runtime permissions needed	Done	Camera and picker through system apps
INTERNET permission removed	Done	`tools:node="remove"` in the manifest
Backup off, data excluded	Done	`allowBackup="false"`; backup and extraction rules
Privacy screen and policy	Done	Privacy & offline screen; `docs/PRIVACY.md`
R8 release build	Done	Minify and resource shrinking with keep rules; verified on the emulator
Signed App Bundle, models in an asset pack	Next	Reduce download size per device
Dark theme and edge-to-edge on all screens	Next	Add a dark colour scheme
Data safety form and store listing	Next	Answers drafted in `docs/PRIVACY.md`

9 Testing and Results

9.1 Unit Tests

Pure Kotlin logic is tested with JUnit on the JVM, so the tests run without a device. The last run of `testDebugUnitTest` executed 38 tests with 0 failures, 0 errors and 0 skipped (Table 9.1).

Table 9.1 Unit tests by class

Test class	Tests	What it checks
CertificateFieldExtractorTest	10	Field rules, abstention on non-certificates, invalid dates
IvpOperationsTest	18	Pipeline Lab image operations
OcrTextCorrectorTest	6	Safe word and digit correction, protected identifiers
CertificateWordPieceTokenizerTest	1	Token ids and character offsets
DocumentKnowledgeIndexTest	1	Retrieval picks the right document
DocumentFormattingTest	1	List formatting of recognised text
ExampleUnitTest	1	Project template test
Total	**38**	**0 failures, 0 errors, 0 skipped**

9.2 Device and Integration Testing

The app was installed and run on an OPPO CPH2665 phone with Android 15, and the R8 release build was verified on the emulator. The screenshots in Chapter 5 were captured on

the emulator with the synthetic certificate and cover every step of the flow: import and crop the photo, preview the flattened page, extract the text and the five fields, save the document, ask when the certificate expires, and open the Privacy & offline screen. The Pipeline Lab with its bundled sample gives a second demonstration path that needs no camera.

9.3 Model Results

The field model was evaluated in Python on text input, using weak validation labels and a six-document, 26-field gold set that was checked by hand and kept out of training (Table 9.2). The corpus behind it has 108 PDFs and 351 pages (295 unique), split by source PDF into 206, 56 and 33 unique pages for training, validation and testing.

Table 9.2 Certificate field model results (text input)

Model	Weak validation	Human-checked gold set
BERT-tiny prototype	35.3% val, 23.5% test exact match	4/26 (15.4%)
MiniLM fine-tuned	35/49 (71.4%)	15/26 (57.7%)
MiniLM int8 ONNX, 32.3 MiB (in the app)	33/49 (67.3%)	15/26 (57.7%)

Per field on the gold set, the fine-tuned model scored 2 of 6 for certificate name, 3 of 6 for issue date, 4 of 4 for port of issue, 2 of 4 for issuing authority and 4 of 6 for expiry date. These are pilot figures on a small set; they are not a claim of 57.7% accuracy on all certificates.

9.4 Not Yet Measured

Three things have not been measured: the accuracy of the whole app from a photographed certificate to five fields, OCR error with ML Kit on the phone, and on-phone latency. The model results above read text, so they do not include capture, cropping or OCR errors. There are no instrumented UI tests yet. A fresh set of photographed certificates with hand-checked fields is the next evaluation step.

10 Project Work and Timeline

10.1 Individual Work

This is a solo project by Aadidev Raizada (N152). The author prepared the certificate dataset of 108 PDFs and 351 pages, with splits by source document and duplicate pages removed; fine-tuned and evaluated the MiniLM model, including building the human-checked gold set; and selected the techniques and directed the build of the application. Claude (Anthropic) was used to implement parts of the application and to draft documentation under the author's direction.

10.2 Timeline

The dates in Table 10.1 come from file and folder modification times, as recorded in `docs/contribution.md`, because the git history was committed in logical layers on 12 October 2026. Each date shows when a piece of work reached its current form.

Table 10.1 Project timeline

Date (2026)	What was built
14 Jul	Android Studio project with Kotlin and Jetpack Compose; first builds on 20 Jul
3 Aug	Camera capture through FileProvider, with no camera permission
7 Aug	Product and design brief
10 Aug	PP-OCRv5 handwriting on ONNX Runtime; layout rebuilt from ML Kit boxes
2 Sep	Document library (JSON and JPEG, atomic writes); MobileBERT Q&A; theme; first unit tests
21 Sep	Corpus of 108 PDFs prepared; bootstrap OCR labels (20,106 line crops); BERT-tiny prototype
23 Sep	Gold set; MiniLM fine-tune (15/26 gold); int8 ONNX export; Kotlin tokenizer; hybrid extraction
12 Oct	Pipeline Lab, unit tests, R8 release build, adaptive icon, backup exclusion, privacy screen and policy

10.3 Problems Solved

Table 10.2 Problems met during development and their solutions

Problem	Solution
ML Kit dependencies request network permissions	Removed INTERNET and ACCESS_NETWORK_STATE with manifest-merger rules
ONNX Runtime needs a file path, not an asset stream	Copy the model to internal storage once with a size check; create the session lazily under a lock
The handwriting model can fail or be unsure	Fall back to ML Kit text for that line
Weak labels flatter validation scores	Report the human-checked gold result (15/26)
Leakage between pages of one certificate	Split by source PDF; remove duplicate pages and text
The model could invent dates on non-certificates	Certificate gate before extraction; model dates must parse as dates
Large app size	R8 and resource shrinking enabled; App Bundle and asset pack planned
Demonstrating without private data	Synthetic certificate generator and a bundled lab sample

11 Conclusion and Future Scope

11.1 Conclusion

NavCert Offline meets its six objectives in one integrated flow. A user can capture a certificate, crop and flatten it, read printed or handwritten text, see five extracted fields, edit and save the document, and ask questions about saved documents, all without a network connection. The network permissions are removed, so documents cannot leave

the phone. The app applies course Units 1 to 3 throughout and part of Unit 7, adds six features beyond the syllabus, follows Material 3, passes 38 unit tests, and completes most of the Google Play release checklist. The field model was evaluated on a hand-checked gold set, and the measurements still missing are stated.

11.2 Future Scope

- A Room (SQLite) document index with full-text search, which would also cover course Unit 6.
- Expiry reminders with local notifications, the most useful feature not yet built.
- A signed App Bundle with the models in an asset pack, the Data safety form and a store listing.
- A dark theme and edge-to-edge layout on every screen.
- Measurement of end-to-end accuracy and latency on the phone with a fresh set of photographed certificates, and instrumented UI tests.

References

[1] BIMCO and International Chamber of Shipping, "Seafarer Workforce Report 2026: Executive summary," Jun. 2026. [Online]. Available: https://www.bimco.org/media/mxhbsjxb/seafarer-workforce-report-2026-executive-summary.pdf

[2] Tokyo MOU Secretariat, *Annual Report on Port State Control in the Asia-Pacific Region 2024*, 2025. [Online]. Available: https://www.tokyo-mou.org/wp/wp-content/uploads/ANN24-1.pdf

[3] M. Schuler, "India cracks down on fraudulent seafarer certificates amid global safety concerns," gCaptain, Jul. 21, 2025. [Online]. Available: https://gcaptain.com/india-cracks-down-on-fraudulent-seafarer-certificates-amid-global-safety-concerns/

[4] International Maritime Organization, "Guidelines on the use of electronic certificates of seafarers," MSC.1/Circ.1665, Jun. 28, 2023.

[5] Google, "Jetpack Compose documentation," Android Developers. Accessed: Oct. 2026. [Online]. Available: https://developer.android.com/develop/ui/compose/documentation

[6] Google, "Material Design 3 in Compose," Android Developers. Accessed: Oct. 2026. [Online]. Available: https://developer.android.com/develop/ui/compose/designsystems/material3

[7] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, Nov. 2000.

[8] Google, "Text recognition v2," ML Kit for Android. Accessed: Oct. 2026. [Online]. Available: https://developers.google.com/ml-kit/vision/text-recognition/v2/android

[9] Microsoft, "ONNX Runtime Mobile," ONNX Runtime documentation. Accessed: Oct. 2026. [Online]. Available: https://onnxruntime.ai/docs/tutorials/mobile/

[10] C. Cui *et al.*, "PP-OCRv5: A specialized 5M-parameter model rivaling billion-parameter vision-language models on OCR tasks," arXiv:2603.24373, Mar. 2026.

[11] Google, "LiteRT overview," Google AI Edge documentation. Accessed: Oct. 2026. [Online]. Available: https://developers.google.com/edge/litert

[12] Z. Sun *et al.*, "MobileBERT: A compact task-agnostic BERT for resource-limited devices," in *Proc. 58th Annu. Meeting Assoc. Computational Linguistics*, 2020, pp. 2158-2170, doi: 10.18653/v1/2020.acl-main.195.

[13] Google, "Data and file storage overview," Android Developers. Accessed: Oct. 2026. [Online]. Available: https://developer.android.com/training/data-storage

[14] Google, "Camera intents," Android Developers. Accessed: Oct. 2026. [Online]. Available: https://developer.android.com/media/camera/camera-intents

[15] Google, "Request runtime permissions," Android Developers. Accessed: Oct. 2026. [Online]. Available: https://developer.android.com/training/permissions/requesting

[16] Google, "Save data in a local database using Room," Android Developers. Accessed: Oct. 2026. [Online]. Available: https://developer.android.com/training/data-storage/room

[17] A. Graves *et al.*, "Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks," in *Proc. 23rd Int. Conf. Machine Learning (ICML)*, 2006, pp. 369-376, doi: 10.1145/1143844.1143891.

[18] W. Wang *et al.*, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," arXiv:2002.10957, 2020.

[19] deepset, "deepset/minilm-uncased-squad2," Hugging Face model card. Accessed: Oct. 2026. [Online]. Available: https://huggingface.co/deepset/minilm-uncased-squad2

[20] P. Rajpurkar, R. Jia, and P. Liang, "Know what you don't know: Unanswerable questions for SQuAD," in *Proc. 56th Annu. Meeting ACL (Short Papers)*, 2018, pp. 784-789, doi: 10.18653/v1/P18-2124.

[21] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF CVPR*, 2018, pp. 2704-2713, doi: 10.1109/CVPR.2018.00286.

[22] J. Canny, "A computational approach to edge detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. PAMI-8, pp. 679-698, 1986, doi: 10.1109/TPAMI.1986.4767851.

[23] K. Zuiderveld, "Contrast limited adaptive histogram equalization," in *Graphics Gems IV*, P. S. Heckbert, Ed. San Diego, CA, USA: Academic Press, 1994, pp. 474-485.

[24] Google, "Material Design 3," m3.material.io. Accessed: Oct. 2026. [Online]. Available: https://m3.material.io/foundations

[25] Google, "Target API level requirements for Google Play apps," Play Console Help. Accessed: Oct. 2026. [Online]. Available: https://support.google.com/googleplay/android-developer/answer/11926878

[26] Google, "Provide information for Google Play's Data safety section," Play Console Help. Accessed: Oct. 2026. [Online]. Available: https://support.google.com/googleplay/android-developer/answer/10787469